5

STATE-SPACE SEARCH

Optimal Search

Optimal Uninformed Search: **Uniform-Cost Search**

n.h.reyes@massey.ac.nz

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path **"length"** measure. Finds "shortest" path.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

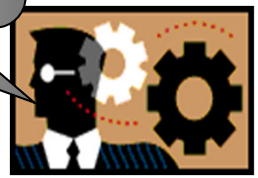
The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

This is what we have seen previously.



Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

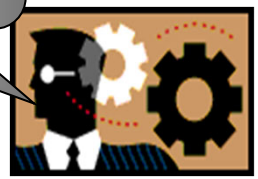
The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; ~~set Visited = (S)~~.
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not ~~in Visited~~ and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; ~~add children of state (N) to Visited~~.
7. Go to Step 2.

Do NOT use Visited List for **Optimal** Searching!



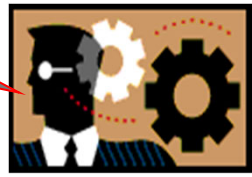
Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Optimal Search

Why not a Visited List?

Why can't we use a Visited List in connection with optimal searching? In the earlier searches, the use of the Visited List guaranteed that we would not do extra work by re-visiting or re-expanding states. It did not cause any failures then (except possibly of intuition)

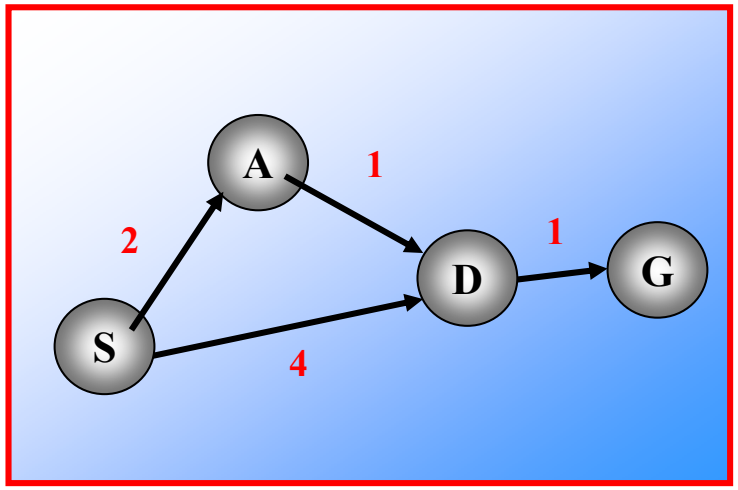


- For the **Any-Path algorithms**, the **Visited List** would not cause us to fail to find a path when one existed, since the path to a state did not matter.

- However, the **Visited List** in connection with **Optimal searches** can cause us to miss the best path.

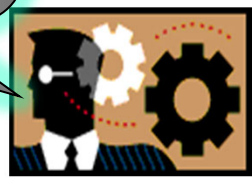
Why not a Visited List?

- For the **Any-Path algorithms**, the Visited List would not cause us to fail to find a path when one existed, since the path to a state did not matter.
- However, the Visited List in connection with **Optimal searches** can cause us to miss the best path.



- The shortest path from S to G is (S A D G).

Clearly, the shortest path (as determined by sum of link costs) to G is (S A D G) and an optimal search had better find it.

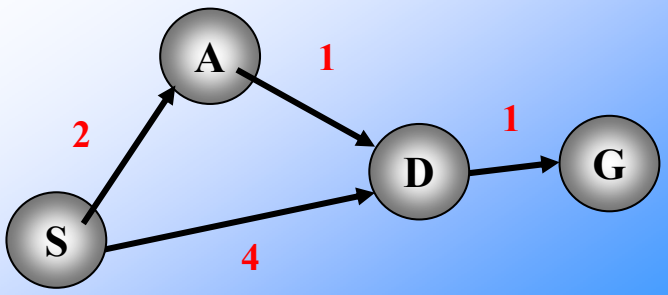


SEARCH

Why not a Visited List?

- For the **Any-Path algorithms**, the Visited List would not cause us to fail to find a path when one existed, since the path to a state did not matter.

- However, the Visited List in connection with **Optimal searches** can cause us to miss the best path.



- But on extending (S), A and D would be added to the Visited List and so (S A) would not be extended to (S A D)

- The shortest path from S to G is (S A D G).

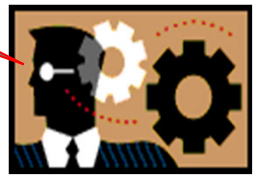
Step	Dequeued	Q	Visited
0		~~(S)~~	S
1	(S)	~~(AS)~~ (DS)	S,A,D
2	(AS)	~~(DS)~~	
3	(DS)	~~(GDS)~~	
4	(GDS)	This is not the shortest path.	

Implementing Optimal Search Strategies

Uniform Cost

- Pick the **best** (**measured by path length**) element of Q.
- Add path extensions **anywhere** in Q.

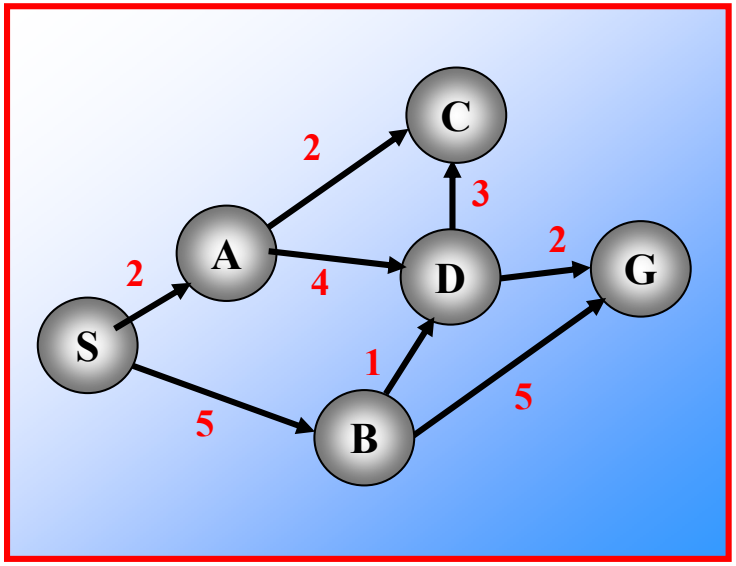
For our convention in class, add the path extensions to the **front** of the Queue.



Implementing Optimal Search Strategies

Uniform Cost

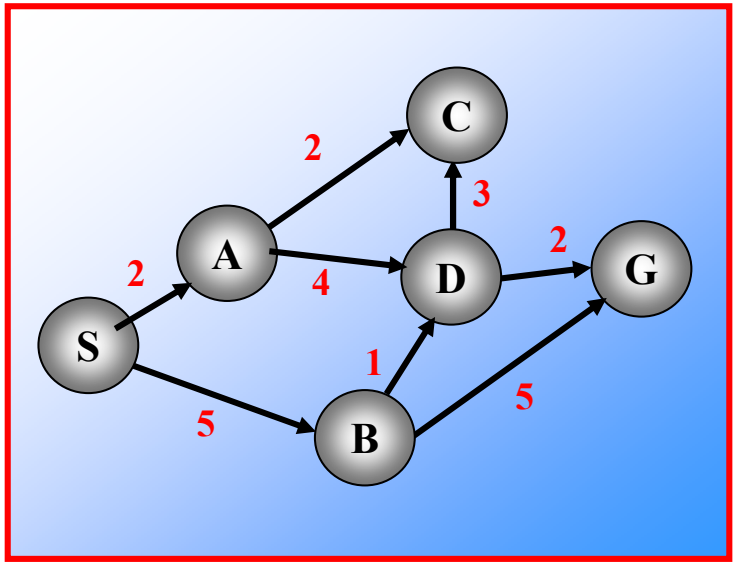
- Like **Best-First Search** except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



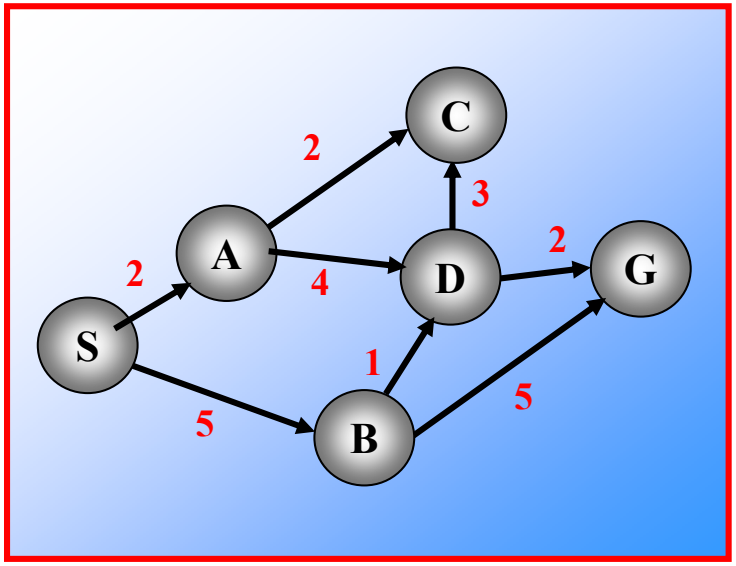
Total path cost:

S-A-C 4

Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



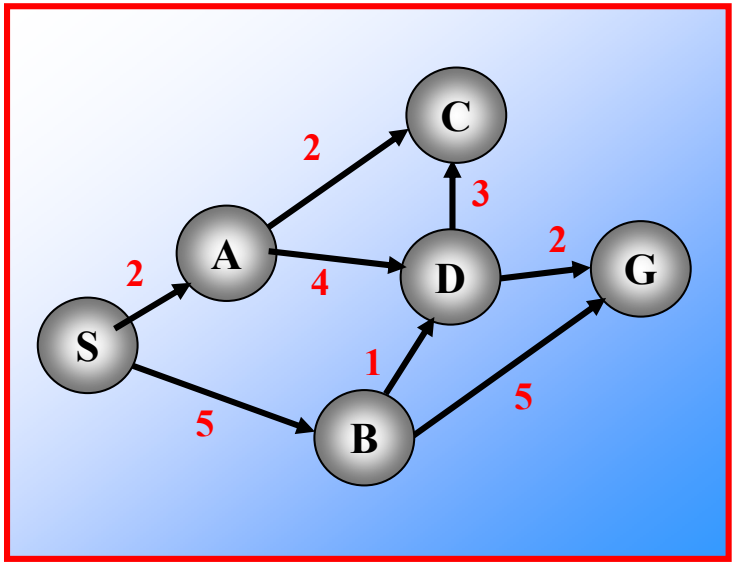
Total path cost:

S-A-C 4
S-B-D-G 8

Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



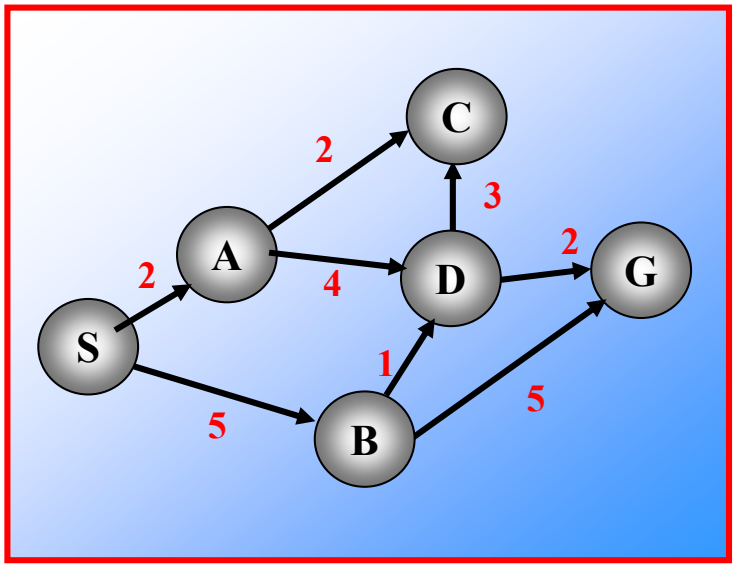
Total path cost:

S-A-C	4
S-B-D-G	8
S-A-D-C	9

SEARCH

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)



Note: the underlined paths are chosen for extension.

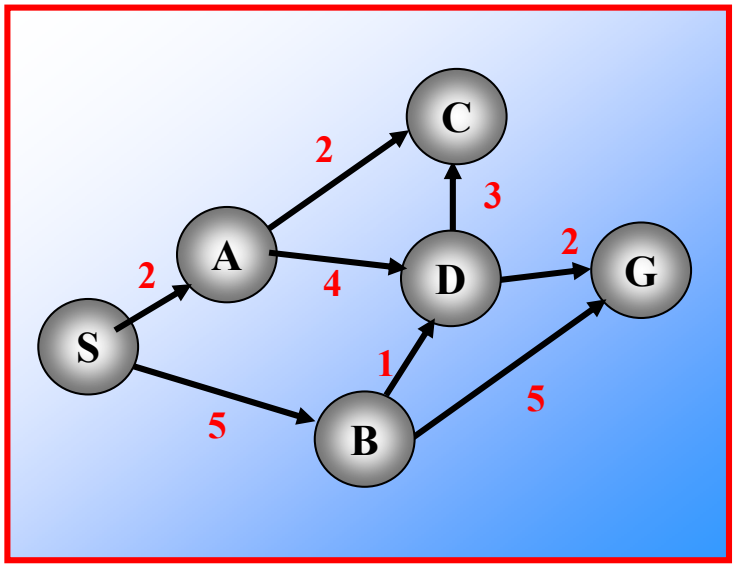
The most recent path is added at the **front of Q** (order is used to settle ties).

Pick the **best** (by path length) element of Q; Add path extensions to **front** of Q.

SEARCH

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS) (8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)



Max Q length = 5
Number of State Expansions = 7
Path = S-A-D-G
Path length = 8

Sequence of State Expansions:
S-A-C-B-D-D-G

Pick the **best** (by path length) element of Q; Add path extensions to **front** of Q.

Why not stop on first visiting a goal?

Uniform Cost

- When doing Uniform Cost, it is **not** correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q. (**look at Step#5 of the simulation earlier**)

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

Why not stop on first visiting a goal?

- When doing Uniform Cost, it is **not** correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q. (**look at Step#5 of the simulation earlier**)

- We must **wait** until such a path is **pulled off the Q** and tested in Step 3. It is only at this point that we are sure it is the shortest path to a goal since there are no other shorter paths that remain unexpanded.

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

Why not stop on first visiting a goal?

Uniform Cost

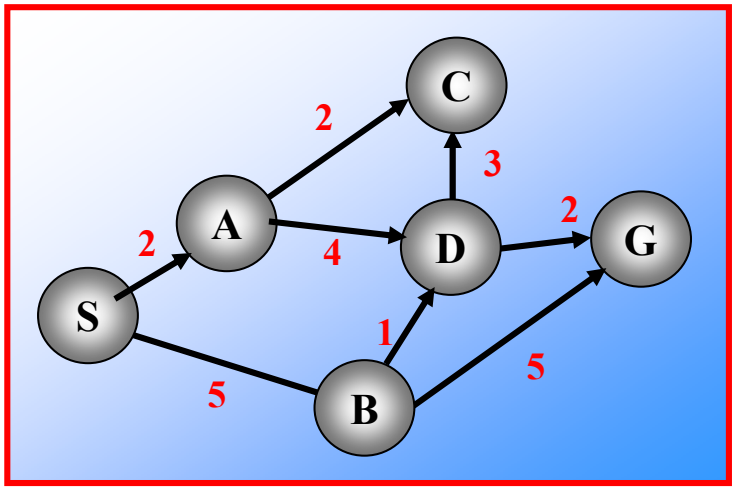
- When doing Uniform Cost, it is not correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q.

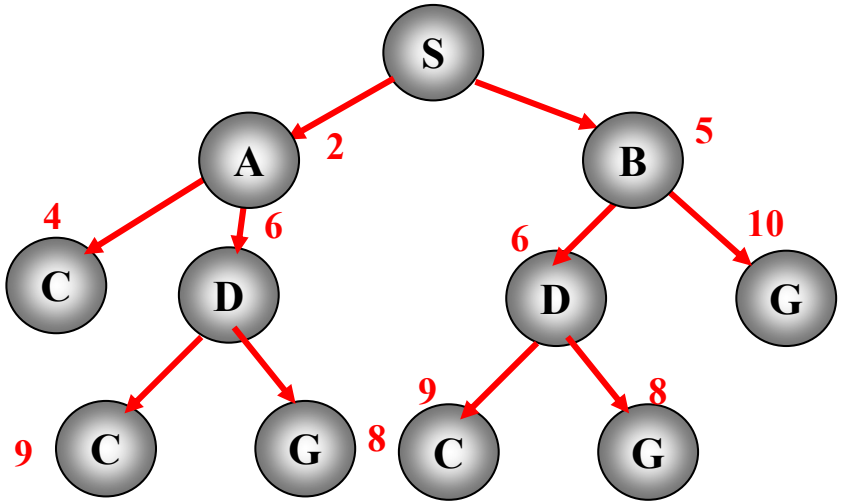
- We must wait until such a path is pulled off the Q and tested in Step 3. It is only at this point that we are sure it is the shortest path to a goal since there are no other shorter paths that remain unexpanded.

- This contrasts with the **non-optimal searches** where the choice of where to test for a goal was a matter of convenience and efficiency, not correctness.

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

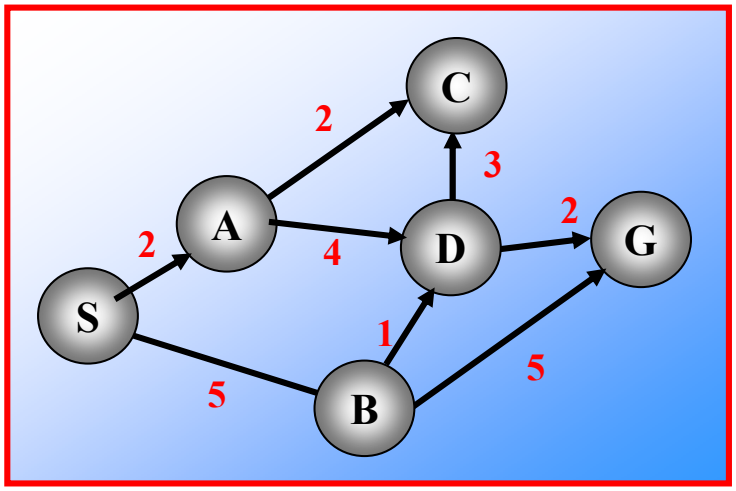


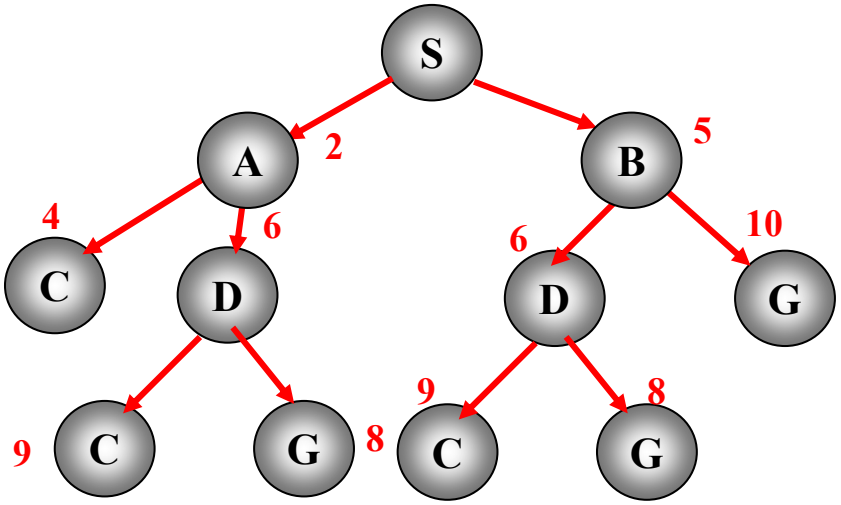


Uniform Cost enumerates paths in order of total path cost!

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)





The sequence of path extensions corresponds precisely to path-length order, so it is not surprising we find the shortest path.

0 2 4 5 6 6 8

Sequence of State Expansions: S – A – C – B – D – D - G

Uniform Cost enumerates paths in order of total path cost!